

Atividade Prática — Containers com Docker

Desenvolvimento de Aplicação Distribuída com Python

Disciplina: Sistemas Operacionais

Tecnologias: Docker, Docker Compose e Python

Objetivo

Desenvolver uma aplicação utilizando containers Docker para simular um ambiente real de microserviços e aplicações distribuídas.

O projeto deverá demonstrar:

- Criação de containers
 - Comunicação entre containers
 - Uso de bancos de dados
 - APIs em Python
 - Docker Compose
 - Redes Docker
 - Persistência de dados
-

Formação dos Grupos

O projeto deverá ser realizado em grupos de até:

3 alunos

Tema do Projeto

O tema será livre.

Cada grupo deverá escolher um problema de negócio e desenvolver uma solução computacional utilizando containers Docker.

Exemplos de Temas

Os grupos poderão desenvolver soluções como:

- Sistema de gerenciamento escolar
- Sistema de controle financeiro
- Sistema de pedidos para restaurante
- Sistema hospitalar
- Sistema de controle de estoque
- Sistema de pet shop
- Sistema de delivery
- Sistema de biblioteca
- Sistema de monitoramento
- Sistema de academia
- Sistema de hotel
- Sistema de oficina mecânica
- Plataforma de cursos
- Sistema de RH

Ou qualquer outro tema aprovado pelo professor.

Requisitos Obrigatórios

O projeto deverá possuir obrigatoriamente:

1. Utilização de Containers Docker

O ambiente deverá possuir no mínimo:

5 containers

Sendo:

Tipo	Quantidade
Banco de Dados	2
Aplicações Python	3

2. Bancos de Dados

Os grupos poderão utilizar bancos como:

- PostgreSQL
- MySQL
- MongoDB
- Redis
- MariaDB

Exemplo:

```
1 container PostgreSQL
1 container MongoDB
```

3. APIs Python

As aplicações deverão ser desenvolvidas em Python.

Sugestões:

- Flask
- FastAPI

As APIs deverão possuir endpoints para:

- Cadastro
 - Consulta
 - Atualização
 - Remoção
-

4. Comunicação Entre Containers

Os containers deverão se comunicar através de rede Docker.

Exemplo:

```
API → Banco PostgreSQL
API → MongoDB
API → Outra API
```

5. Docker Compose

O ambiente deverá obrigatoriamente utilizar:

```
docker-compose.yml
```

ou

```
compose.yaml
```

Estrutura Esperada do Projeto

Exemplo:

```
projeto/
├── api-clientes/
│   ├── app.py
│   └── Dockerfile
├── api-produtos/
│   ├── app.py
│   └── Dockerfile
├── api-pedidos/
│   ├── app.py
│   └── Dockerfile
├── docker-compose.yml
└── README.md
```

Exemplo de Arquitetura

O projeto poderá possuir uma arquitetura semelhante:

```
[API Clientes] → PostgreSQL
[API Produtos] → MongoDB
[API Pedidos]  → Comunicação com APIs anteriores
```

Exemplo de Containers Esperados

Container	Função
api-clientes	API Python
api-produtos	API Python
api-pedidos	API Python
postgres-db	Banco PostgreSQL
mongo-db	Banco MongoDB

Requisitos Técnicos

O projeto deverá demonstrar:

- Uso correto de Dockerfile
 - Uso de Docker Compose
 - Exposição de portas
 - Variáveis de ambiente
 - Persistência de dados
 - Comunicação entre containers
 - Execução correta do ambiente completo
-

Entregas

Cada grupo deverá entregar:

Código-fonte completo

Incluindo:

- Dockerfiles
- docker-compose.yml
- APIs
- Scripts